IE410: Introduction to Robotics

Project Part A

Pick-and-Place Operations with the Arduino Braccio Robot Arm

Course	IE410: Introduction to Robotics
Semester	Winter 2026
Group	Group 18
Students	Yagnik Patel (202401160@dau.ac.in)
	Dharmraj Vaghela (202401236@dau.ac.in)
	Nirmal Nakum (202401022@dau.ac.in)
	Harshit Goyal (202401065@dau.ac.in)
	Arnav Pandita (202401079@dau.ac.in)

Table of Contents

1. Introduction

Robots are being used in many places these days — factories, hospitals, warehouses, and even homes. One of the most common things a robot arm does is pick an object up from one location and place it somewhere else. This is called a **pick-and-place** operation.

In this project, we used the **Arduino Braccio** robotic arm to carry out two types of pick-and-place tasks. The first task used pre-written (hardcoded) positions, and the second task used a camera to find the object first and then pick it up.

The main goal was to get hands-on experience with how a robotic arm moves, how it grabs things, and how sensors and cameras can help make the robot smarter.

2. Project Objectives

The main goals of this project were:

- Understand how a robotic arm moves using servo motors.
- Learn how a pick-and-place task works in practice.
- Program the robot to follow a fixed sequence of movements.
- Use a camera to detect an object and guide the robot toward it.
- Use an IR sensor to check whether the object was successfully grabbed.
- Understand how two robot arms can work together (handover task).

3. Hardware and Software Used

Component	Purpose
Arduino Uno/Mega	Main controller – runs the robot program
Braccio Robot Arm	6-DOF robotic arm with two-finger gripper
Servo Motors (×6)	Control each joint of the arm
IR Sensor	Detects if the object is inside the gripper
Webcam / Smartphone	Detects object position for Task 2
Arduino IDE	Used to write and upload the code
Serial Communication	Sends error values from camera to Arduino

4. Task 1: Pre-Programmed Pick and Place

In this task, the robot picks up an object (like an eraser or a small block) from a fixed location called **Point A** and places it at another fixed location called **Point B**. All positions are written directly in the code — the robot does not use any sensors to find the object.

4.1 Sequence of Movement

Step	What Happens	Key Angles (Base, Shoulder, Elbow, WV, WR, Gripper)
1 – Approach	Arm moves above Point A	120, 80, 170, 90, 70, 10 (open)
2 – Align	Arm lowers to object level	120, 80, 170, 90, 70, 10
3 – Grip	Gripper closes around object	120, 80, 170, 90, 70, 80 (closed)
4 – Lift	Arm raises the object up	120, 100, 120, 10, 150, 85
5 – Transport	Arm swings toward Point B	100, 100, 120, 10, 150, 15

The gripper is set to **10 degrees** when open and around **73–80 degrees** when fully closed. Small delays between steps help the arm move smoothly without jerking.

5. Task 2: Camera-Assisted Pick and Place

In this task, the object can be placed anywhere on the workspace — its position is not fixed. A camera is used to find the object, and the robot adjusts its position based on what the camera sees.

5.1 How It Works

- **Camera detects the object** and calculates how far it is from the centre of the frame (error in X and Y).
- **Error values are sent via Serial** to the Arduino as a formatted message, e.g., *<12.5, -8.3>*.
- **The robot nudges itself** using proportional control — a small correction based on the error size.
- **Once aligned**, the camera sends a *<GRASP>* command to trigger the grasping sequence.
- **The IR sensor checks** if the object is actually inside the gripper before lifting.
- **If confirmed**, the robot lifts the object and drops it at a predefined Point B.

During the implementation of Task 2, the vision system and object detection were working successfully. The camera was able to detect the ball and estimate its position correctly. However, the **M2 motor (shoulder motor)** of the Braccio robotic arm was not functioning properly during testing. Due to this hardware issue, the robot arm was unable to physically reach the detected object even though the detection and control logic were operating correctly. Therefore, the perception and detection part of the task was successfully completed, but the full pick-and-place execution was limited because of the motor failure.

6. Working Principle

Below is the complete step-by-step flow of how the robot operates in Task 2:

Step 1 – Object Detection

The camera looks at the workspace and finds the object using colour detection or a marker. It then calculates how far the object is from the centre of the camera frame.

Step 2 – Robot Alignment

Error values (err_x, err_y) are sent to Arduino via Serial. The robot makes small angular adjustments to the base and shoulder to centre itself over the object.

Step 3 – Grasping

Once the error is small enough, the camera sends the GRASP command. The arm lowers and the gripper closes.

Step 4 – IR Sensor Check

The IR sensor checks if the beam inside the gripper is broken — this confirms an object is being held. If not, the grasp is considered a failure and the robot returns to hover.

Step 5 – Lifting

If the grasp is confirmed, the arm raises the object upward by increasing shoulder angle.

Step 6 – Transport to Point B

The arm rotates to the drop-off location (predefined angles for the base, shoulder, elbow, etc.).

Step 7 – Releasing

The gripper opens (10 degrees) and the object is released at Point B.

Step 8 – Return to Hover

The robot returns to its starting hover position, ready for the next object.

7. Code Explanation

ServoMovement(step, M1, M2, M3, M4, M5, M6)

This is the main Braccio library function. It moves all six servo motors at once. The first argument is the step delay (10–30 ms) which controls how smoothly the arm moves. M1 = base, M2 = shoulder, M3 = elbow, M4 = wrist vertical, M5 = wrist rotation, M6 = gripper.

executeStep(m1, m2, m3, m4, m5, m6)

This is a small helper function that calls ServoMovement and then waits 1 second. It keeps the code cleaner — instead of writing the same delay every time, we just call executeStep().

```
void executeStep(int m1, ..., int m6) {
Braccio.ServoMovement(20, m1, m2, m3, m4, m5, m6);
```

```
delay(1000);
}
```

nudgeRobot(err_x, err_y)

This function moves the robot slightly based on the error from the camera. It multiplies the error by a small gain (Kp) to calculate how much to change each joint angle. This is a simple proportional controller — bigger error = bigger correction.

```
current_base += err_x * Kp_Base;
current_shoulder += err_y * Kp_Reach;
```

executeTactileGrasp()

This is the main grasping function. It lowers the arm, reads the IR sensor, closes the gripper if an object is detected, lifts it up, moves to Point B, releases it, and returns home.

IR Sensor Usage

The IR sensor is connected to pin A0. When an object is between the gripper fingers, it breaks the IR beam and the digitalRead() returns LOW. This confirms a successful grasp.

```
int beamState = digitalRead(IR_SENSOR_PIN);
if (beamState == LOW) { /* object detected */ }
```

Serial Communication

The camera (running on a PC/phone) sends messages over USB serial. The Arduino reads them with Serial.readStringUntil(). Messages like <5.2,-3.1> trigger nudging, and triggers the grasping sequence.

```
String data = Serial.readStringUntil('\n');
if (data == "<GRASP>") { executeTactileGrasp(); }
```

8. Observations and Results

After running both tasks, here is what we observed:

- The robot successfully picked up and placed objects in Task 1 with good accuracy.
- In Task 2, camera assistance allowed the robot to work even when the object was placed at different spots.
- Some small alignment errors occurred due to lighting changes affecting the camera.
- The IR sensor was very helpful — it prevented the robot from trying to place an object it never actually grabbed.
- Servo delays were important — too fast and the arm would jerk or miss the object.
- The proportional controller (nudgeRobot) worked well for small corrections but was slow for large errors.

9. Challenges Faced

- **Servo Alignment:** Each servo has a slightly different zero position. Getting them all aligned correctly at the start took a lot of trial and error.
- **Gripper Accuracy:** The gripper sometimes closed too early or too late. We had to fine-tune the angle values and add extra delays to make it reliable.
- **Sensor Calibration:** The IR sensor was sensitive to ambient light. We had to shield it slightly and test the threshold carefully.
- **Camera Positioning:** The camera needed to be fixed at a specific height and angle to give correct error values. Even a small shift in the camera position changed the readings.
- **Motion Timing:** Finding the right delay between movements was tricky. Too short and servos clashed; too long and the task took forever.

10. Conclusion

This project gave us a really good introduction to how robotic arms work in the real world. We started with a simple pre-programmed pick-and-place task and then moved on to using a camera and sensor feedback to make the robot smarter.

We learned how servo angles control arm positions, how proportional control can adjust robot movement in real time, and how sensors like IR can add a layer of reliability to grasping tasks.

The main takeaway is that even a 'simple' task like picking up a block involves a lot of careful planning — timing, sensor integration, camera calibration, and joint coordination all have to work together. This kind of practical experience is hard to get just from lectures, and working with the Braccio arm made the concepts from IE410 much clearer.

- Gained practical experience programming a 6-DOF robot arm.
- Understood how to integrate camera and sensor feedback into robot control.
- Learned the basics of proportional control and serial communication.
- Understood the challenges of real-world robotics versus theoretical models.